Live Streaming Speech-to-Text

Technical Design Document

1. Purpose & Scope

This document defines the technical design of each component in the live STT pipeline — internal structure, method signatures, data contracts, and key engineering decisions.

Stack: Python 3.11 · FastAPI · faster-whisper · Silero VAD · Vanilla JS frontend.

2. Project Structure

```
stt-system/  client/    audio_capture.js       # Component 1
websocket_client.js    # Component 2    transcript_ui.js       # Component 8
index.html  server/    main.py                # FastAPI app + startup
session_manager.py     # Component 3    vad_engine.py          # Component 4
audio_processor.py     # Component 5    asr_model.py           # Component 6
post_processor.py      # Component 7    config.py              # All tunable
constants  tests/  requirements.txt
```

Design principle: One file per component. All tunable constants live in config.py — never hardcoded in logic files.

3. Component Technical Designs

Component 1 — Audio Capture Module

Layer	File	Language
Frontend	`client/audio_capture.js`	JavaScript

Purpose: Capture raw PCM audio from the microphone and emit fixed-size binary chunks to the WebSocket transport.

Properties

stream	MediaStream from getUserMedia
audioContext	AudioContext — resamples input to 16kHz
processor	ScriptProcessorNode — fires every FRAME_SIZE samples
onChunk	Callback(Int16Array) — called with each 20ms frame
isCapturing	Boolean guard — prevents double-start

Methods

`constructor(onChunk)` *stores callback, deferred start*	Stores chunk callback. Does not open the mic — explicit start() required.
`async start()` *returns: Promise<void>*	Calls getUserMedia. Creates AudioContext at 16kHz. Connects mic → ScriptProcessorNode. Throws on permission denied — caller handles.
`stop()` *returns: void*	Disconnects processor, stops media tracks, closes AudioContext. Safe to call multiple times.
`_onAudioProcess(event)` *private · fires every 20ms*	Reads float32 buffer, converts to Int16Array via _float32ToInt16(), calls onChunk(). No allocations, no async in this path.
`_float32ToInt16(buffer)` *private · pure · no mutation*	Multiplies by 32767, clamps to [-32768, 32767], returns new Int16Array.

Config constants

SAMPLE_RATE	16000 Hz
FRAME_SIZE	320 samples (20ms at 16kHz)
CHANNELS	1 (mono)

Output contract → Component 2

Int16Array of exactly 320 samples (640 bytes) emitted every 20ms via onChunk callback.

Key design decision

float32→int16 conversion is done client-side. This halves wire payload (4 bytes/sample → 2 bytes/sample) and keeps the server pipeline uniform regardless of browser.

Component 2 — WebSocket Transport

Layer	File	Language
Transport	`websocket_client.js` / `main.py`	JS (client) · Python (server)

Purpose: Maintain a persistent bidirectional channel. Audio bytes upstream as binary. Transcript JSON downstream as text.

Client — WebSocketClient

`constructor(url, onTranscript)` *deferred connect*	Stores URL and transcript callback. Does not connect — explicit connect() required.
`connect()` *returns: void*	Opens WebSocket. Attaches event handlers. Sends session_id on open if resuming.

`sendChunk(int16Array)` *called by Component 1*	Guards socket is OPEN, then socket.send(int16Array.buffer). Raw binary — no encoding.
`_onMessage(event)` *private*	JSON.parse(event.data), calls onTranscript(). Malformed messages logged and skipped.
`_scheduleReconnect()` *private · backoff*	Retries at 1s, 2s, 4s. After 3 failures, surfaces fatal error and stops.
`disconnect()` *clean close 1000*	Sends close frame, nulls socket, resets retryCount.

Server — main.py

`@app.on_event('startup')` *runs once*	Loads Whisper model and Silero VAD into app.state. Server refuses to start if either fails.
`@app.websocket('/ws')` *one call per connection*	Accepts connection, creates SessionManager, spawns receive_loop and send_loop as concurrent asyncio tasks.
`receive_loop(ws, session)` *async · lifetime of connection*	Loops: await ws.receive_bytes() → session.audio_queue.put(). Never processes audio — only moves bytes.
`send_loop(ws, session)` *async · lifetime of connection*	Loops: await session.transcript_queue.get() → json.dumps() → ws.send_text(). Only place that writes to the socket.

Key design decision — two-loop pattern

receive_loop and send_loop run concurrently and neither processes audio. The socket stays always reading and always writing, independent of inference duration. Without this, a 400ms Whisper call would freeze the socket.

Backpressure

audio_queue has max depth 50 frames (1s). If full, oldest frame is dropped and a warning event is emitted to the client.

Component 3 — Session Manager

Layer	File	Language
Backend	`server/session_manager.py`	Python

Purpose: Own all per-connection state. All other backend components are stateless — this is the single object holding the running context.

State

session_id	UUID4 — sent to client on connect, used to resume after drop
audio_queue	asyncio.Queue (max 50) — receives bytes from receive_loop
transcript_queue	asyncio.Queue — receives results from post-processor, consumed by send_loop
ring_buffer	np.ndarray int16, pre-allocated 30s — circular, oldest data overwritten
vad_state	VadState enum: SILENCE | SPEECH | PAUSED
silence_frame_count	Consecutive silent frames since last speech
speech_frames	List[bytes] — accumulates speech frames, flushed on segment end
partial_text	Most recent interim transcript for the active segment
segments	List[Segment] — finalised segments, append-only

Methods

`__init__(self)` *called once per connection*	Generates session_id, creates both queues, pre-allocates ring_buffer as numpy zeros.
`async pipeline_loop(vad, asr, post)` *main processing loop*	Dequeues frame → writes to ring_buffer → VAD classify → state machine → on flush: audio_processor → ASR in thread → post-processor → transcript_queue.
`_handle_vad_result(is_speech)` *private · state machine*	Advances VAD state machine. Returns VADAction: ACCUMULATE, FLUSH, PAUSE, or CONTINUE.
`_flush_segment()` *private*	Concatenates speech_frames into numpy array, clears list, records segment_start_time. Returns audio array for ASR.
`add_final_segment(segment)` *called after ASR*	Appends Segment dataclass to self.segments. Resets partial_text.
`teardown()` *on disconnect*	Puts None sentinel on audio_queue to exit pipeline_loop.

VAD state machine

From	To	Trigger	Action
`SILENCE`	`SPEECH`	Speech detected	Start accumulating. Record segment_start_time.
`SPEECH`	`SILENCE`	300ms silence (15 frames)	Flush speech_frames → ASR.
`SILENCE`	`PAUSED`	5s silence (250 frames)	Pause pipeline. Emit pause event to client.
`PAUSED`	`SPEECH`	Speech detected	Resume pipeline. Emit resume event to client.

Supporting types

VadState(Enum)	SILENCE | SPEECH | PAUSED — enum prevents invalid state assignment
Segment (dataclass)	segment_id, text, start, end, confidence — frozen, immutable after creation

---

Component 4 — VAD Engine

Layer	File	Language
Backend	`server/vad_engine.py`	Python

Purpose: Classify each 20ms audio frame as speech or silence. Stateless — all session state lives in Component 3.

Properties

model	Silero VAD (torch.nn.Module) — injected, shared read-only across sessions
sample_rate	16000 — must match config
threshold	0.5 — confidence cutoff, tunable in config

Methods

`__init__(model, threshold)` *dependency injected*	Stores model and threshold. Model loaded in main.py, not here.
`classify(frame: bytes) → bool` *synchronous · called every 20ms*	Calls _preprocess() → model inference → threshold comparison. Never raises — exceptions return False.
`_preprocess(frame) → Tensor` *private*	bytes → np.int16 → float32 (÷32768) → torch.Tensor shape (1, 320).
`get_confidence(frame) → float` *dev/tuning use*	Returns raw model confidence instead of bool. Used to tune threshold value.

Threshold guide

Value	Behaviour	Notes
`0.3`	Too sensitive	Keyboard clicks and noise classified as speech. Garbage output.
`0.5`	Default · balanced	Works in quiet to moderate noise. Recommended starting point.
`0.7`	Conservative	Good for noisy environments. Risk: soft speech missed at segment starts.
`0.9`	Too strict	Misses first syllable of most utterances. Studio-quality audio only.

Note: 3 consecutive speech frames (60ms) required before a segment starts — enforced in Component 3. Prevents single noise bursts from triggering ASR.

Component 5 — Audio Processor

Layer	File	Language
Backend	`server/audio_processor.py`	Python

Purpose: Transform accumulated speech frames into a float32 numpy array satisfying Whisper's input contract. Stateless and pure.

Methods

`process(frames, input_rate) → ndarray` *main public method*	Orchestrates: _assemble → _to_float32 → _resample (conditional) → _normalise → _validate. Raises AudioProcessingError on failure.
`_assemble(frames) → ndarray` *step 1*	np.frombuffer each frame, np.concatenate into single int16 array shape (N,).
`_to_float32(audio) → ndarray` *step 2*	Divides int16 by 32768.0. Returns float32 in [-1.0, 1.0]. Does not mutate input.
`_resample(audio, input_rate) → ndarray` *step 3 · conditional*	Runs only if input_rate ≠ 16000. Uses scipy.signal.resample.
`_normalise(audio) → ndarray` *step 4*	Peak normalisation — divides by max absolute value. Skips if max < 0.001 to avoid amplifying silent noise.
`_validate(audio) → None` *step 5 · raises on failure*	Asserts: dtype=float32, shape is 1D, duration 0.3s–30s. Raises AudioProcessingError with failing condition.

Transformation summary

Input	List[bytes] — e.g. 75 frames × 640 bytes
After _assemble	np.ndarray int16, shape (24000,) — 1.5s at 16kHz
After _to_float32	float32, range [-1.0, 1.0]
After _resample	Shape adjusted to 16kHz if input rate differed
After _normalise	Max absolute value = 1.0
Output	float32 ndarray · 16kHz · 1D · duration 0.3–30s

Supporting type

AudioProcessingError	Caught in Component 3 — segment discarded, error event emitted to client, session continues.

Component 6 — ASR Model

Layer	File	Language
Model	`server/asr_model.py`	Python

Purpose: Convert a preprocessed audio array into a raw text transcript with timestamps and confidence scores. Most computationally expensive component in the system.

Properties

model	faster-whisper WhisperModel — injected, shared read-only across sessions
executor	ThreadPoolExecutor(max_workers=2) — offloads blocking inference from the asyncio event loop
language	ISO code e.g. 'en'. None = auto-detect (+50ms overhead).
beam_size	5 — decoder beam width. Higher = more accurate, slower.
word_timestamps	True — per-word start/end times. ~10% overhead.
timeout	2.0s — max inference time per segment. Exceeded → empty result.

Methods

`__init__(model, executor, ...)` *dependency injected*	Stores all injected resources. Nothing loaded here.
`async transcribe(audio) → ASRResult` *primary public method*	Offloads _run_inference() to thread pool via run_in_executor. Wraps with asyncio.wait_for(timeout=2.0). On timeout → returns empty ASRResult with is_timeout=True.
`_run_inference(audio) → ASRResult` *private · runs in thread*	Calls model.transcribe(). Materialises segment generator with list(). Assembles text, word timestamps, confidence.
`_build_result(segments, info) → ASRResult` *private · pure assembly*	Joins segment texts. Flattens word timestamps. Computes average confidence from no_speech_prob.

Model size guide

Model	Params	Latency*	Recommendation
`base`	74M	~150ms	Start here. Good accuracy in quiet conditions.
`small`	244M	~300ms	Production balance. Handles accents and moderate noise.
`medium`	769M	~700ms	High accuracy. Approaches 2s timeout on long segments (CPU only).
`large-v3`	1.5B	~1.5s+	Near human-level. GPU required for real-time use.

** CPU-only, 2s audio segment (Intel i7). GPU reduces all values by 5–10×.*

Supporting types

ASRResult (dataclass)	text, words: List[WordTimestamp], confidence, duration, is_timeout — frozen
WordTimestamp (dataclass)	word, start, end, probability — per-word timing for UI highlighting

Key design decision — run_in_executor

Whisper is CPU-bound. Without run_in_executor, every inference call blocks the event loop for 150–700ms — no other connection is served during this window. Offloading to a thread keeps the loop free.

Thread pool sizing: max_workers=2. More workers causes CPU cache contention. On GPU, set max_workers=1 — inference is serialised anyway.

Component 7 — Post-Processor

Layer	File	Language
Model	`server/post_processor.py`	Python

Purpose: Transform raw ASR output into clean, readable text. Last processing stage before transcript leaves the server. Stateless and pure.

Note: Filler words (um, uh, er) are preserved — not stripped.

Properties

itn_processor	nemo_text_processing Processor — injected at construction
punct_model	deepmultilingualpunctuation Model | None — optional
use_punct_model	bool from config. False = regex rules. Start False for practice build.

Methods

`process(result: ASRResult) → TranscriptMessage` *main public method*	Runs: _capitalise → _apply_itn → _apply_punctuation → _build_message. Never raises — step failure returns raw text unchanged.
`_capitalise(text) → str` *step 1*	Uppercases text[0] only.
`_apply_itn(text) → str` *step 2*	Converts spoken-form numbers, dates, currencies to written form. Falls back to _apply_itn_regex() if nemo unavailable.

`_apply_punctuation(text) → str` *step 3*	ML model if use_punct_model=True, else regex rules. Appends period if text ends without punctuation.
`_apply_itn_regex(text) → str` *fallback · ordered regex*	Rules applied in order: currency → ordinal dates → cardinals → percentages → times → sentence 'I'. Order is load-bearing.
`_build_message(text, result, type) → TranscriptMessage` *step 4*	Assembles TranscriptMessage from processed text and original ASRResult metadata.

ITN transformation examples

"fifty dollars"	"$50"
"fourteenth of march"	"14th of March"
"three meetings"	"3 meetings"
"forty percent"	"40%"
"three thirty pm"	"3:30 PM"

Supporting type

TranscriptMessage (dataclass)	type, text, segment_id, start, end, confidence, words. to_dict() returns JSON-serialisable plain dict.

Component 8 — Transcript UI

Layer	File	Language
Frontend	`client/transcript_ui.js`	JavaScript

Purpose: Render interim and final transcript segments in the browser with minimal flicker.

Properties

container	HTMLElement — scrollable transcript div. aria-live='polite' set on construction.
interimEl	Single persistent span — updated in-place on every interim event. Never recreated.
finalSegments	Map<segment_id, HTMLElement> — tracks all finalised paragraphs. Prevents duplicate rendering.
autoScroll	True by default. Set to false if user scrolls up — respects their scroll position.

Methods

`constructor(containerId)` *sets up DOM*	Finds container, creates interimEl span, attaches scroll listener, sets aria-live.
`handleMessage(message)` *single dispatch*	Routes by message.type → renderInterim(), renderFinal(), updateVadIndicator(), or renderError().
`renderInterim(message)` *in-place update*	Sets interimEl.textContent = message.text. Reuses existing span — no element creation, no reflow.
`renderFinal(message)` *appends permanent element*	Guards against duplicate segment_id. Creates p element, inserts before interimEl, clears interim text.
`updateVadIndicator(message)` *status dot*	Updates indicator: green (speech), grey (silence), amber (paused).
`_scrollToBottom()` *conditional*	Sets scrollTop = scrollHeight only if autoScroll is true.
`clear()` *resets UI*	Removes all final paragraphs, clears interimEl and finalSegments map.

Key design decision — in-place interim render

Interim events arrive every ~200ms. Creating/removing a DOM element at that frequency causes layout reflows and visible flicker. A single persistent span updated via textContent is a targeted text node mutation — no reflow, no flicker.

CSS classes

.transcript-container	overflow-y: auto · max-height: 60vh · aria-live: polite
.transcript-final	color: text-primary · permanent · no animation
.transcript-interim	color: text-tertiary · italic · blinking cursor via ::after
.vad-indicator	12px dot · CSS transition on background-color

---

4. Cross-Cutting Concerns

Error handling policy

Component	Policy
Audio Capture (C1)	getUserMedia failure propagates to caller. UI layer handles permission errors.
WebSocket (C2)	WebSocketDisconnect is normal exit. Unexpected drop → exponential backoff reconnect.
Session Manager (C3)	Segment < 0.3s discarded before ASR. Sentinel (None) used for teardown.
VAD Engine (C4)	Any exception → log + return False. Session continues.
Audio Processor (C5)	Validation failure → AudioProcessingError. Caught in C3, segment discarded.

ASR Model (C6)	Timeout → empty ASRResult (is_timeout=True). Runtime error → segment discarded.
Post-Processor (C7)	Any step failure → log + return raw text unchanged. Never drops a segment.
Transcript UI (C8)	handleMessage wrapped in try/catch. Malformed message logged, session continues.

Dependency injection

Whisper, Silero VAD, and ITN processor are loaded once at server startup and injected via constructor parameters. No component loads its own dependencies. Enables unit testing with mocks and eliminates per-connection startup cost.

Stateless components

Components 4, 5, and 7 hold no per-session state. They are instantiated once and shared safely across all concurrent sessions. All session state is owned exclusively by Component 3.

config.py — all tunable constants

SAMPLE_RATE	16000
FRAME_SIZE	320
VAD_THRESHOLD	0.5
VAD_FLUSH_FRAMES	15 (300ms)
VAD_PAUSE_FRAMES	250 (5s)
ASR_TIMEOUT	2.0s
ASR_BEAM_SIZE	5
WS_QUEUE_MAX	50
MIN_SEGMENT_DURATION	0.3s
MAX_SEGMENT_DURATION	30.0s